MODUL VII
AUTENTIKASI DAN OTORISASI PENGGUNA

I. Tujuan Praktikum

1. Mahasiswa memahami konsep keamanan perangkat lunak (security) sebagai bagian dari kebutuhan non-fungsional dan atribut kualitas perangkat lunak.
2. Mahasiswa mampu membedakan serta mengimplementasikan mekanisme Autentikasi (Authentication) dan Otorisasi (Authorization).
3. Mahasiswa mampu menerapkan teknik enkripsi satu arah (password hashing) menggunakan pustaka hashlib di Python.
4. Mahasiswa mampu menerapkan pembatasan hak akses berbasis peran (Role-Based Access Control / RBAC) pada antarmuka aplikasi desktop MVC.

II. Alat dan Bahan

1. PC/Laptop
2. Visual Studio Code
3. Laragon
4. Python 3.x

III. Teori Dasar

Dalam rekayasa perangkat lunak, kualitas sebuah sistem tidak hanya dinilai dari kelengkapan fitur fungsionalnya (seperti kemampuan menyimpan atau menampilkan data), tetapi juga dari pemenuhan kebutuhan non-fungsional (non-functional requirements). Salah satu atribut kualitas non-fungsional yang paling krusial dalam arsitektur perangkat lunak adalah keamanan (security). Sistem informasi perpustakaan yang baik harus mampu melindungi integritas data koleksi buku dan informasi anggotanya dari akses maupun manipulasi oleh pihak yang tidak berwenang.

Autentikasi (Authentication) dan Otorisasi (Authorization) sering disebut bersamaan, autentikasi dan otorisasi adalah dua proses keamanan yang berbeda dan bekerja secara berurutan:

1. Autentikasi (Authentication)

   Proses memverifikasi identitas pengguna yang mencoba masuk ke dalam sistem (menjawab pertanyaan: "Siapa Anda?"). Mekanisme paling umum adalah memeriksa

kecocokan kombinasi username dan password yang dimasukkan pengguna pada halaman Login terhadap data yang tersimpan di basis data.

2. Otorisasi (Authorization)

   Proses menentukan hak akses atau izin tindakan yang boleh dilakukan oleh pengguna setelah identitasnya berhasil diverifikasi (menjawab pertanyaan: "Apa yang boleh Anda lakukan?"). Misalnya, setelah berhasil login, pengguna dengan peran (role) Admin/Petugas diizinkan menambah dan menghapus buku, sedangkan pengguna dengan peran Anggota hanya diizinkan melihat daftar katalog buku (read-only).



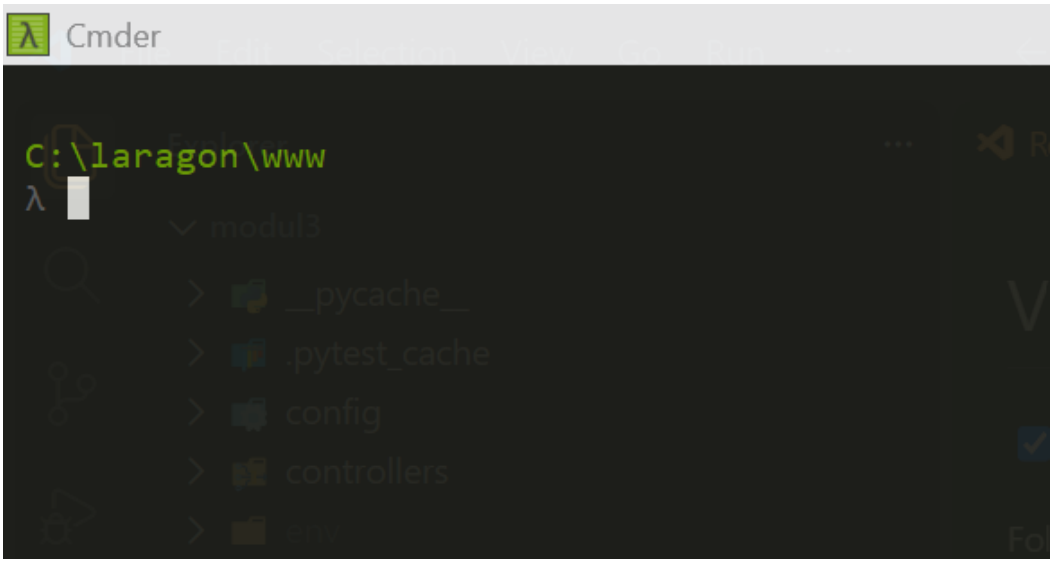
Saat menyimpan kata sandi (password) pengguna secara langsung dalam bentuk teks biasa (plain text) di dalam tabel MySQL merupakan celah keamanan fatal. Jika basis data mengalami kebocoran (data breach), seluruh kata sandi pengguna akan langsung terbaca. Untuk mengatasi risiko tersebut, standar rekayasa perangkat lunak mewajibkan penggunaan fungsi Cryptographic Hash (seperti algoritma SHA-256). Hashing adalah proses mengubah teks kata sandi menjadi deretan karakter acak dengan panjang tetap secara satu arah (one-way function). Nilai yang sudah di-hash tidak dapat dikembalikan menjadi teks asli (irreversible). Saat pengguna melakukan login, sistem akan melakukan hashing terhadap input kata sandi yang diketik, lalu membandingkan nilai hash tersebut dengan nilai hash yang tersimpan di basis data.

Penerapan otorisasi berbasis peran (Role-Based Access Control) di dalam pola arsitektur Model-View-Controller (MVC) didistribusikan sesuai tanggung jawab masing-masing komponen:

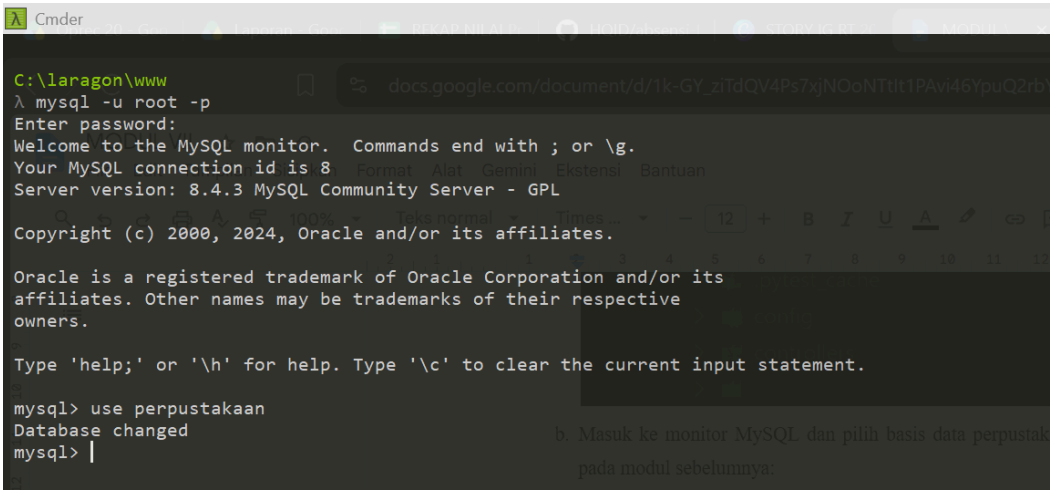
1. Model: Bertugas melakukan kueri verifikasi kredensial ke basis data menggunakan Parameterized Query (untuk mencegah SQL Injection) dan mengembalikan informasi peran (role) pengguna.
2. Controller: Menerima status autentikasi beserta role dari Model, lalu mengatur logika alur aplikasi—apakah pengguna dialihkan ke jendela utama aplikasi atau ditolak masuk.
3. View: Menyesuaikan tampilan antarmuka pengguna berdasarkan instruksi Controller, seperti menonaktifkan (disable) atau menyembunyikan tombol manipulasi data bagi pengguna yang tidak memiliki hak akses.

IV. Prosedur Kerja

1. Pembuatan Tabel users pada Basis Data MySQL
   a. Buka aplikasi Laragon, pastikan layanan MySQL telah berjalan, lalu buka menu Terminal.

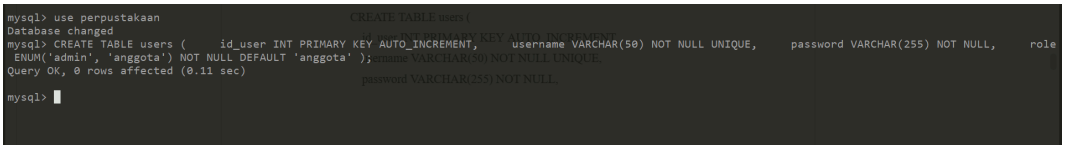


   b. Masuk ke monitor MySQL dan pilih basis data perpustakaan yang telah dibuat pada modul sebelumnya:
   mysql -u root -p
   USE perpustakaan;



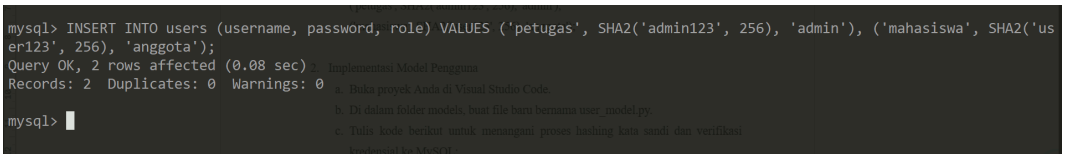
c. Buat tabel users untuk menyimpan kredensial dan role pengguna dengan kueri DDL berikut:

```
CREATE TABLE users (
    id_user INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'anggota') NOT NULL DEFAULT 'anggota'
);
```



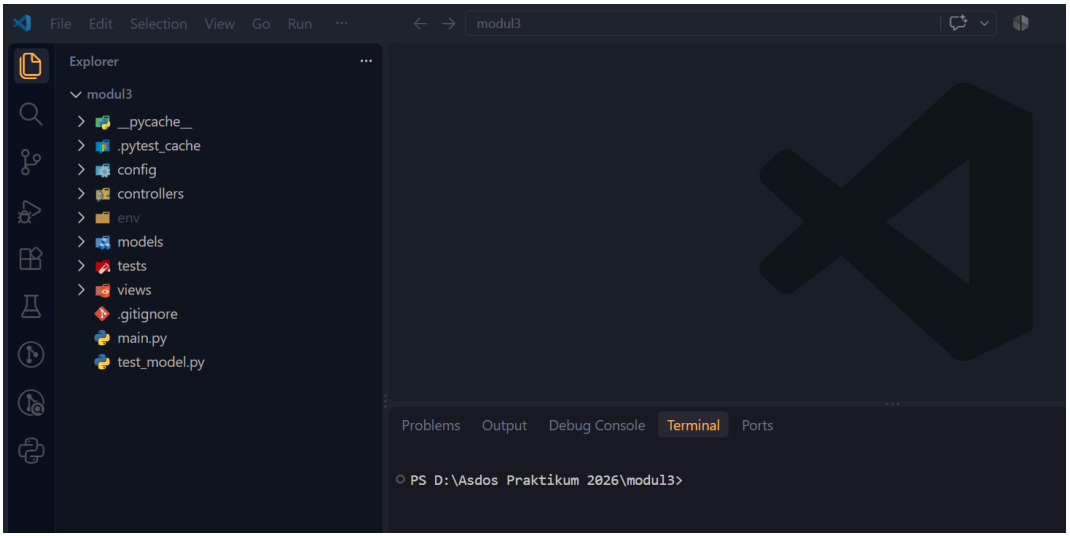
d. Masukkan 2 akun uji coba (1 Admin dan 1 Anggota) menggunakan fungsi SHA2(..., 256) bawaan MySQL agar kata sandi langsung tersimpan dalam format hash SHA-256:

```
INSERT INTO users (username, password, role) VALUES
('petugas', SHA2('admin123', 256), 'admin'),
('mahasiswa', SHA2('user123', 256), 'anggota');
```

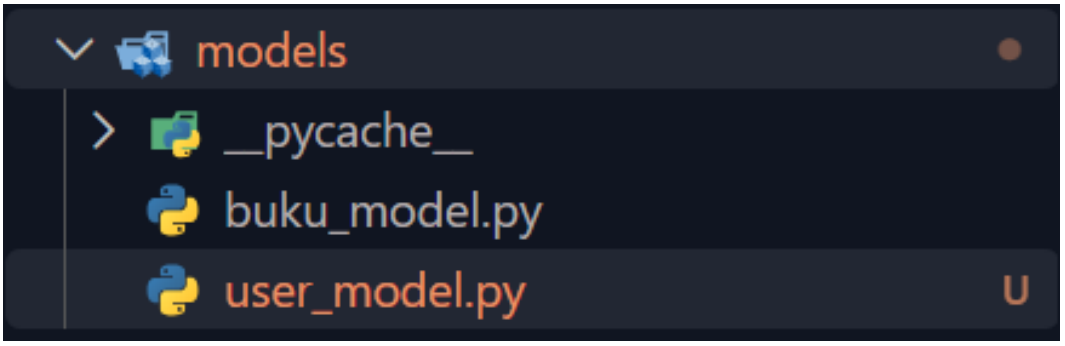


2. Implementasi Model Pengguna

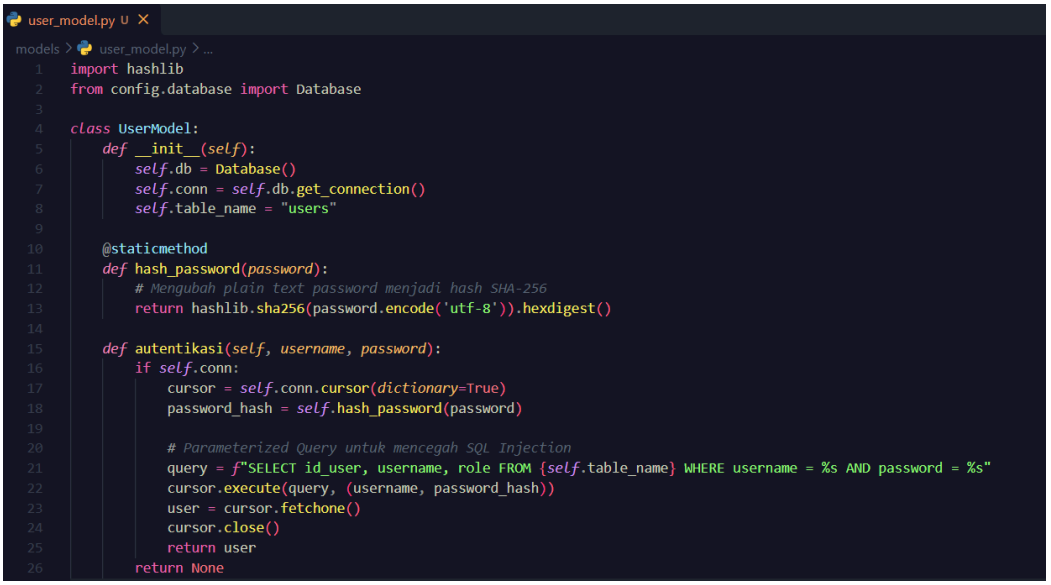
a. Buka proyek Anda di Visual Studio Code.



b. Di dalam folder models, buat file baru bernama user_model.py.

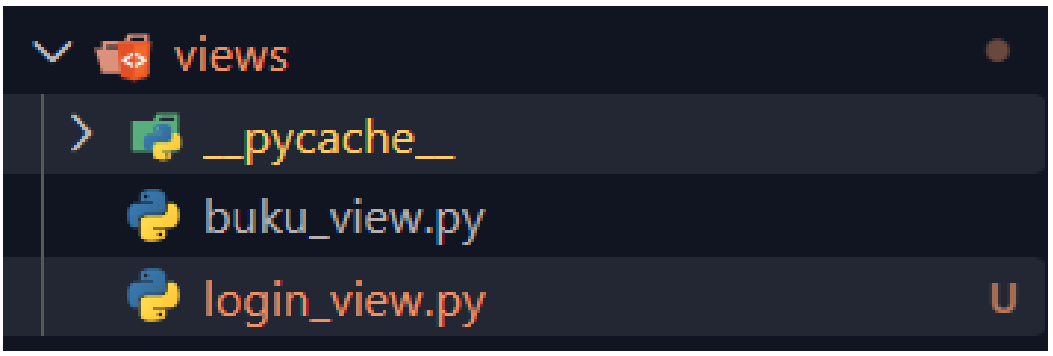


c. Tulis kode berikut untuk menangani proses hashing kata sandi dan verifikasi kredensial ke MySQL:



3. Pembuatan Antarmuka Login

a. Di dalam folder views, buat file baru bernama login_view.py.



b. Tulis kode berikut untuk merancang jendela Login:



4. Penerapan Otorisasi pada BukuController

a. Buka file controllers/buku_controller.py dan modifikasi konstruktor __init__ agar menerima parameter user_role dan membatasi akses tombol berdasarkan role:

buku_controller.py M

controllers > buku_controller.py > ...

```python
import tkinter.messagebox as messagebox
from models.buku_model import BukuModel
from views.buku_view import BukuView

class BukuController:
    def __init__(self, user_role="anggota"):
        self.user_role = user_role
        self.model = BukuModel()
        self.view = BukuView()

        # Menampilkan informasi role pada judul jendela aplikasi
        self.view.title(f"Sistem Manajemen Perpustakaan - Mode: {self.user_role.upper()}")

        # Binding tombol simpan
        self.view.btn_simpan.configure(command=self.simpan_data)

        # OTORISASI: Jika bukan admin, nonaktifkan form input dan tombol simpan
        if self.user_role != "admin":
            self.view.btn_simpan.configure(state="disabled", text="Akses Terkunci (Khusus Admin)")
            self.view.entry_judul.configure(state="disabled")
            self.view.entry_penulis.configure(state="disabled")
            self.view.entry_tahun.configure(state="disabled")

        self.muat_data_tabel()

    def muat_data_tabel(self):
        for item in self.view.tabel.get_children():
            self.view.tabel.delete(item)
        data_buku = self.model.get_all_buku()
        for buku in data_buku:
            self.view.tabel.insert("", "end", values=(buku['id_buku'], buku['judul'], buku['penulis'], buku['tahun_terbit']))
```

```python
    def simpan_data(self):
        # Proteksi ganda di level Controller
        if self.user_role != "admin":
            messagebox.showerror("Akses Ditolak", "Anda tidak memiliki izin untuk menambah data!")
            return

        judul = self.view.entry_judul.get()
        penulis = self.view.entry_penulis.get()
        tahun = self.view.entry_tahun.get()

        if not judul or not penulis or not tahun:
            messagebox.showwarning("Peringatan", "Semua kolom form harus diisi!")
            return

        if self.model.create_buku(judul, penulis, tahun):
            messagebox.showinfo("Sukses", "Data buku berhasil disimpan!")
            self.view.entry_judul.delete(0, 'end')
            self.view.entry_penulis.delete(0, 'end')
            self.view.entry_tahun.delete(0, 'end')
            self.muat_data_tabel()
        else:
            messagebox.showerror("Error", "Gagal menyimpan data.")

    def run(self):
        self.view.mainloop()
```

5. Implementasi AuthController dan Pembaruan main.py
    a. Buat file baru di dalam folder controllers bernama auth_controller.py:

auth_controller.py U

controllers > auth_controller.py > ...

```python
import tkinter.messagebox as messagebox
from models.user_model import UserModel
from views.login_view import LoginView
from controllers.buku_controller import BukuController

class AuthController:
    def __init__(self):
        self.model = UserModel()
        self.view = LoginView()
        self.view.btn_login.configure(command=self.proses_login)

    def proses_login(self):
        username = self.view.entry_username.get()
        password = self.view.entry_password.get()

        if not username or not password:
            messagebox.showwarning("Peringatan", "Username dan Password wajib diisi!")
            return

        user = self.model.autentikasi(username, password)

        if user:
            messagebox.showinfo("Login Berhasil", f"Selamat datang, {user['username']} ({user['role']})!")
            # Tutup jendela Login
            self.view.destroy()
            # Buka jendela Utama Perpustakaan dengan mengirimkan role pengguna
            app_utama = BukuController(user_role=user['role'])
            app_utama.run()
        else:
            messagebox.showerror("Login Gagal", "Username atau Password salah!")

    def run(self):
        self.view.mainloop()

```

b. Buka file main.py di root proyek Anda, lalu ubah titik masuk utama agar memanggil AuthController terlebih dahulu:

main.py M

main.py > ...

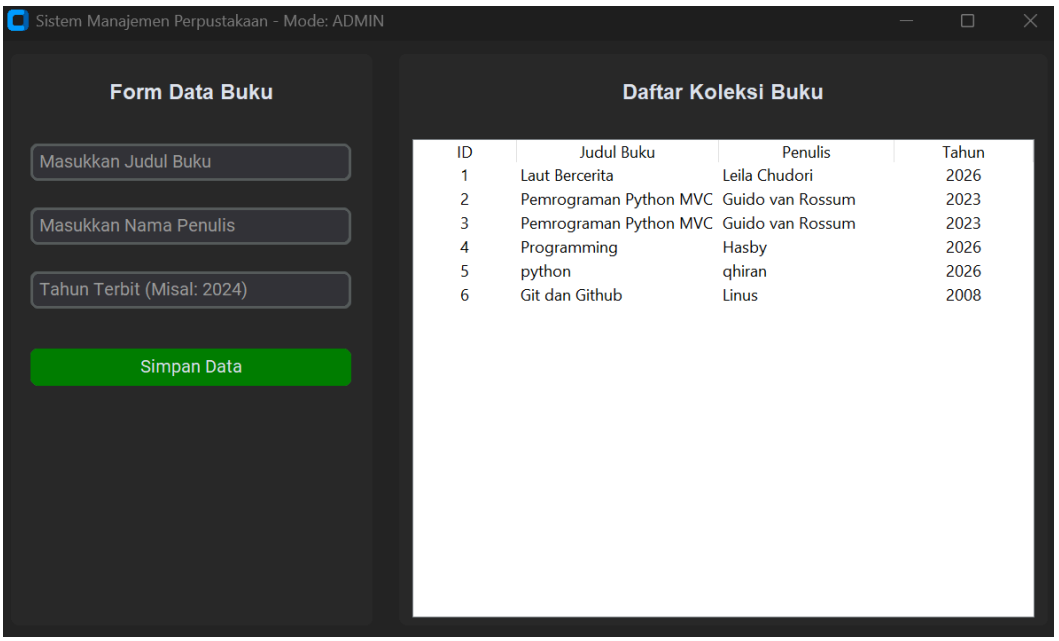
You, 6 days ago | author (You)

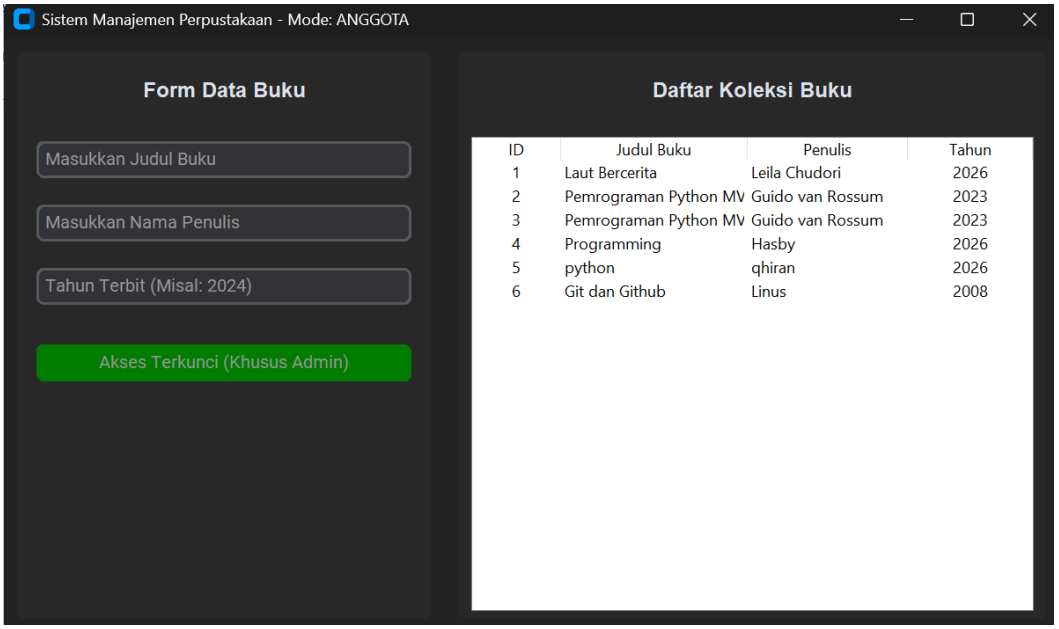
```python
from controllers.auth_controller import AuthController

if __name__ == "__main__":
    app = AuthController()
    app.run()

```

c. Jalankan python main.py di terminal. Uji coba masuk menggunakan akun petugas (password: admin123) dan akun mahasiswa (password: user123)

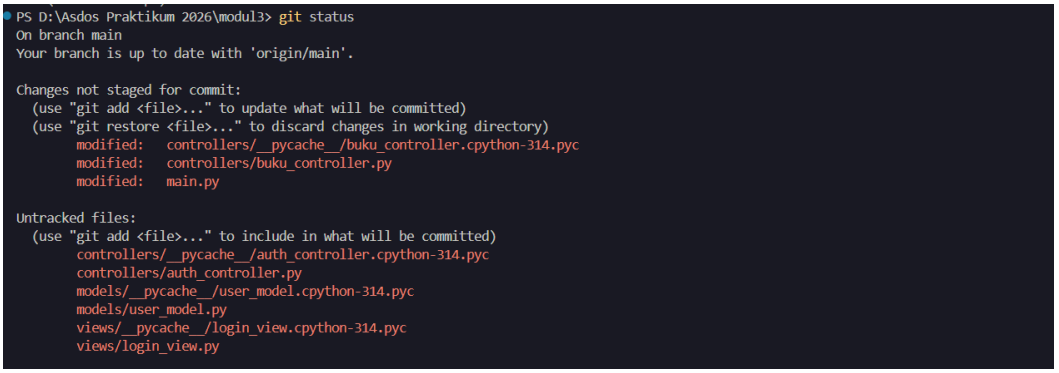




6. Menyimpan Perubahan ke Repositori Git

   a. Periksa daftar file yang berubah di terminal

   git status



b. Tambahkan seluruh file ke staging area:

git add .

```
PS D:\Asdos Praktikum 2026\modul3> git add .
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
```

c. Lakukan commit dengan pesan deskriptif:

git commit -m "feat: implementasi autentikasi login dan otorisasi role pengguna"

```
PS D:\Asdos Praktikum 2026\modul3> git commit -m "feat: implementasi autentikasi login dan otorisasi role pengguna"
[main 649581e] feat: implementasi autentikasi login dan otorisasi role pengguna
 9 files changed, 110 insertions(+), 27 deletions(-)
 create mode 100644 controllers/__pycache__/auth_controller.cpython-314.pyc
 create mode 100644 controllers/auth_controller.py
 create mode 100644 models/__pycache__/user_model.cpython-314.pyc
 create mode 100644 models/user_model.py
 create mode 100644 views/__pycache__/login_view.cpython-314.pyc
 create mode 100644 views/login_view.py
```

d. Unggah perubahan ke repositori GitHub Anda:

git push origin main

```
PS D:\Asdos Praktikum 2026\modul3> git push origin main
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 18 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (17/17), 8.43 KiB | 411.00 KiB/s, done.
Total 17 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/HQID/modul3.git
   3d24f1a..649581e  main -> main
```

V. Hasil dan Pembahasan

1. Tampilan tabel users di MySQL dengan kolom password telah terenkripsi hash SHA-256.
2. Tampilan Login saat dijalankan
3. Tampilan halaman Buku sebagai admin
4. Tampilan halaman Buku sebagai anggota

TUGAS

1. Tambahkan metode register_user(username, password, role) pada kelas UserModel (models/user_model.py) yang secara otomatis mengubah password menjadi hash SHA-256 sebelum disimpan ke tabel users.
2. Terapkan juga aturan otorisasi pada tombol "Perbarui Data (Update)" dan "Hapus Data (Delete)" yang telah Anda buat pada modul sebelumnya, sehingga kedua tombol

tersebut ikut terkunci (state="disabled") apabila pengguna masuk dengan role anggota.

3. Tambahkan tombol "Logout" pada BukuView (views/buku_view.py) yang apabila diklik akan menutup jendela perpustakaan dan mengembalikan pengguna ke jendela LoginView.